

ここでは、基本的な構文要素と例をまとめています。各セクションの最後には、関連トピックの詳細な説明へのリンクがあります。

また、JetBrains Academyが提供する無料の[Kotlin Core](#) [トラック](#)を利用すれば、Kotlinの基礎をすべて学ぶことができます。

Package definition and imports

パッケージの定義とインポート

パッケージの仕様は、ソースファイルの先頭に記述する必要があります：

```
package my.demo

import kotlin.text.*

// ...
```

ディレクトリとパッケージを一致させる必要はありません。ソースファイルはファイルシステム上の任意の場所に配置できます。

See [Packages](#).

Program entry point

プログラムのエントリポイント（入口・始まりの場所）を

Kotlin アプリケーションのエントリポイントは `main` 関数です：

```
fun main() {
    println("Hello world!")
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-hello-world"}

`main` の別の形式では、可変個の `String` 引数を受け取ります：

```
fun main(args: Array<String>) {
    println(args.contentToString())
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3"}

Print to the standard output

標準出力への表示

`print` は、その引数を標準出力に出力します：

```
fun main() {  
    //sampleStart  
    print("Hello ")  
    print("world!")  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-print"}

`println` は引数を表示し、改行を挿入するため、次に表示する内容は次の行に表示されます：

```
fun main() {  
    //sampleStart  
    println("Hello world!")  
    println(42)  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-println"}

Read from the standard input

標準入力から読み込む

`readln()` 関数は、標準入力から読み込みを行います。この関数は、ユーザーが入力した行全体を文字列として読み込みます。

`println()`、`readln()`、`print()` 関数を組み合わせて使用することで、ユーザーからの入力を要求したり、入力結果を表示したりするメッセージを出力することができます：

```
// 入力を求めるメッセージを表示します  
println("Enter any word: ")  
  
// ユーザー入力を受け取り、保存します。例：Happiness  
val yourWord = readln()  
  
// 入力内容を含むメッセージを表示します  
print("You entered the word: ")  
print(yourWord)  
// You entered the word: xxxxxxxx
```

詳細については、[標準入力の読み取り](#)を参照してください。

Functions

関数

2つの `Int` パラメータを持ち、戻り値の型が `Int` である関数：

```
//sampleStart
fun sum(a: Int, b: Int): Int {
    return a + b
}
//sampleEnd

fun main() {
    print("sum of 3 and 5 is ")
    println(sum(3, 5))
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-return-int"}

関数の本体は式であってもよい。その戻り値の型は推論される：

```
//sampleStart
fun sum(a: Int, b: Int) = a + b
//sampleEnd

fun main() {
    println("sum of 19 and 23 is ${sum(19, 23)}")
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-function-expression"}

意味のある値を返さない関数：

```
//sampleStart
fun printSum(a: Int, b: Int): Unit {
    println("sum of $a and $b is ${a + b}")
}
//sampleEnd

fun main() {
    printSum(-1, 8)
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-return-unit"}

`Unit` の戻り値の型は省略可能です：

```
//sampleStart
fun printSum(a: Int, b: Int) {
    println("sum of $a and $b is ${a + b}")
}
//sampleEnd

fun main() {
    printSum(-1, 8)
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-function-omit-unit"}

See [Functions](#).

Variables

変数

Kotlin では、変数を宣言する際は、キーワード **val** または **var** で始め、その後に変数名を続けます。

val キーワードを使用して、一度だけ値が代入される変数を宣言します。これらは不変の読み取り専用ローカル変数(書き換え不可能なローカル変数)であり、初期化後は別の値を再代入することはできません：

```
fun main() {
//sampleStart
    // 書き換え不可能な変数 x を宣言し、値 5 を代入する
    val x: Int = 5
    // 5
//sampleEnd
    println(x)
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-val"}

再代入可能な変数(書き換え可能な)を宣言するには、**var** キーワードを使用します。これらは可変変数であり、初期化後にその値を変更することができます：

```
fun main() {
//sampleStart
    // 書き換え可能な変数 x を宣言し、値 5 を代入する
    var x: Int = 5
    // 変数 x に新しい値 6 を代入します
    x += 1
    // 6
//sampleEnd
    println(x)
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-var"}
```

Kotlinは型推論をサポートしており、宣言された変数のデータ型を自動的に特定します。変数を宣言する際、変数名の後の型を省略することができます：

```
fun main() {  
    //sampleStart  
    // 変数 x を値 5 で宣言します。型 `Int` が推論されます。  
    val x = 5  
    // 5  
    //sampleEnd  
    println(x)  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-inference"}
```

変数は、初期化してからでないと使用できません。変数は、宣言時に初期化することも、まず宣言しておいて後で初期化することもできます。2番目のケースでは、データ型を指定する必要があります：

```
fun main() {  
    //sampleStart  
    // 宣言時に変数 x を初期化します。この場合は型指定は不要です  
    val x = 5  
    // 変数 c を初期化せずに宣言します。型指定は必須です。  
    val c: Int  
    // 宣言後に変数 c を初期化します  
    c = 3  
  
    //sampleEnd  
    println(x) // 5  
    println(c) // 3  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-initialize"}
```

変数は最上位レベルで宣言できます：

```
//sampleStart  
val PI = 3.14  
var x = 0  
  
fun incrementX() {  
    x += 1  
}  
// x = 0; PI = 3.14  
// incrementX()  
// x = 1; PI = 3.14  
//sampleEnd
```

```
fun main() {
    println("x = $x; PI = $PI")
    incrementX()
    println("incrementX()")
    println("x = $x; PI = $PI")
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-variable-top-level"}

プロパティの宣言に関する詳細については、[プロパティ](#)を参照してください。

Creating classes and instances

クラスとインスタンスの作成

クラスを定義するには、`class` キーワードを使用します：

```
class Shape
```

クラスのプロパティは、その宣言または本体に記述することができます。

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
```

クラス宣言に列挙されたパラメータを持つデフォルトコンストラクタは、自動的に利用可能です：

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
fun main() {
    val rectangle = Rectangle(5.0, 2.0)
    println("The perimeter is ${rectangle.perimeter}")
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-class-constructor"}

クラス間の継承は、コロン（`:`）を用いて宣言されます。クラスはデフォルトで `final` です。クラスを継承可能にするには、そのクラスを `open` として指定します：

```
open class Shape

class Rectangle(val height: Double, val length: Double): Shape() {
```

```
    val perimeter = (height + length) * 2
}
```

For more information about constructors and inheritance, see [Classes](#) and [Objects and instances](#).

Comments

コメント

ほとんどの現代のプログラミング言語と同様に、Kotlin では 1 行（または 行末）コメントと複数行（ブロック）コメントがサポートされています：

```
// これは行末のコメントです

/* これはブロックコメントです
   複数行にまたがって。 */
```

Kotlin のブロックコメントはネストすることができます：

```
/* The comment starts here
   /* contains a nested comment */
   and ends here. */
```

ドキュメントコメントの構文については、[Kotlin コードのドキュメント化](#) を参照してください。

String templates

文字列テンプレート

```
fun main() {
    //sampleStart
    var a = 1
    // テンプレート内の単純な名前：
    val s1 = "a is $a"

    a = 2
    // テンプレート内の任意の式：
    val s2 = "${s1.replace("is", "was")}, but now is $a"
    //sampleEnd
    println(s2)
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-string-templates"}

詳細については、[文字列テンプレート](#)を参照してください。

Conditional expressions

条件式

```
//sampleStart
fun maxOf(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}
//sampleEnd

fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-conditional-expressions"}

Kotlin では、`if` を式として使うこともできます：

```
//sampleStart
fun maxOf(a: Int, b: Int) = if (a > b) a else b
//sampleEnd

fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-if-expression"}

See [if-expressions](#).

for loop

forループ

```
fun main() {
    //sampleStart
    val items = listOf("apple", "banana", "kiwifruit")
    for (item in items) {
        println(item)
    }
    //sampleEnd
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-for-loop"}
```

または:

```
fun main() {  
    //sampleStart  
    val items = listOf("apple", "banana", "kiwifruit")  
    for (index in items.indices) {  
        println("item at $index is ${items[index]}")  
    }  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-for-loop-indices"}
```

See [for loop](#).

while loop

whileループ

```
fun main() {  
    //sampleStart  
    val items = listOf("apple", "banana", "kiwifruit")  
    var index = 0  
    while (index < items.size) {  
        println("item at $index is ${items[index]}")  
        index++  
    }  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-while-loop"}
```

See [while loop](#).

when expression

when 式

```
//sampleStart  
fun describe(obj: Any): String =  
    when (obj) {  
        1 -> "One"  
        "Hello" -> "Greeting"  
        is Long -> "Long"  
        !is String -> "Not a string"  
        else -> "Unknown"
```

```
}  
//sampleEnd  
  
fun main() {  
    println(describe(1))  
    println(describe("Hello"))  
    println(describe(1000L))  
    println(describe(2))  
    println(describe("other"))  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-when-expression"}

See [when expressions and statements](#).

Ranges

範囲（レンジ）

Check if a number is within a range using `in` operator:

```
fun main() {  
    //sampleStart  
    val x = 10  
    val y = 9  
    if (x in 1..y+1) {  
        println("fits in range")  
    }  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-range-in"}

数値が範囲外かどうかを確認する：

```
fun main() {  
    //sampleStart  
    val list = listOf("a", "b", "c")  
  
    if (-1 !in 0..list.lastIndex) {  
        println("-1は範囲外です。")  
    }  
    if (list.size !in list.indices) {  
        println("リストのサイズも、有効なリストインデックスの範囲外になっています。")  
    }  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-out-of-range"}
```

範囲内をひとつずつ処理する：

```
fun main() {  
    //sampleStart  
    for (x in 1..5) {  
        print(x)  
    }  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-iterate-range"}
```

あるいは、範囲を一定の間隔に従って処理する：

```
fun main() {  
    //sampleStart  
    for (x in 1..10 step 2) {  
        print(x)  
    }  
    println()  
    for (x in 9 downTo 0 step 3) {  
        print(x)  
    }  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-iterate-progression"}
```

See [Ranges and progressions](#).

Collections

コレクション（集合）

コレクションを繰り返しに処理する：

```
fun main() {  
    val items = listOf("apple", "banana", "kiwifruit")  
    //sampleStart  
    for (item in items) {  
        println(item)  
    }  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-iterate-collection"}
```

in 演算子を使用して、コレクションにオブジェクトが含まれているかどうかを確認します：

```
fun main() {  
    val items = setOf("apple", "banana", "kiwifruit")  
    //sampleStart  
    when {  
        "orange" in items -> println("juicy")  
        "apple" in items -> println("apple is fine too")  
    }  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-collection-in"}
```

ラムダ式 を使用して、コレクションのフィルタリングやマップ処理を行います：

```
fun main() {  
    //sampleStart  
    val fruits = listOf("banana", "avocado", "apple", "kiwifruit")  
    fruits  
        .filter { it.startsWith("a") }  
        .sortedBy { it } // ソート  
        .map { it.uppercase() } // 大文字化  
        .forEach { println(it) } // ひとつずつ表示  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-collection-filter-map"}
```

See [Collections overview](#).

Nullable values and null checks

ヌル許容値とヌルチェック

null 値が可能な場合、参照型は明示的に null 許容型として指定する必要があります。Null許容型の名前は、末尾に **?** が付きます。たとえば、**Int?** などです。

str が整数でない場合は **null** を返す：

```
fun parseInt(str: String): Int? {  
    return str.toIntOrNull()  
}
```

null 許容値を返す関数を使用します：

```

fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}

//sampleStart
fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1) // 文字列を引数に取り、パース(文字列の構造解析が)可能な数値
    // を整数として返します。
    val y = parseInt(arg2)

    // `x * y` を使用すると、両方に NULL が含まれている可能性があるため、エラーが発生し
    // ます。
    if (x != null && y != null) {
        // nullチェックの後、x と y は自動的に null 許容型にキャストされます
        println(x * y)
    }
    else {
        println("' $arg1 ' or ' $arg2 ' is not a number")
    }
}
//sampleEnd

fun main() {
    printProduct("6", "7")
    printProduct("a", "7")
    printProduct("a", "b")
}

```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-function-nullable-value"}

または、:

```

fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}

fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1)
    val y = parseInt(arg2)

    //sampleStart
    // ...
    if (x == null) {
        println("arg1 の数値形式が不正です: ' $arg1 '")
        return
    }
    if (y == null) {
        println("arg2 の数値形式が不正です: ' $arg2 '")
        return
    }
}

```

```

        // nullチェックの後、x と y は自動的に null 許容型にキャストされます
        println(x * y)
    //sampleEnd
}

fun main() {
    printProduct("6", "7")
    printProduct("a", "7")
    printProduct("99", "b")
}

```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-function-null-check"}

See [Null-safety](#).

Type checks and automatic casts

型チェックと自動型変換

`is` 演算子は、式がある型のインスタンスであるかどうかを調べます。不変のローカル変数やプロパティが特定の型であるかどうかを確認する場合、明示的に型変換を行う必要はありません：

```

//sampleStart
fun getStringLength(obj: Any): Int? {
    if (obj is String) {
        // この分岐では、`obj` は自動的に `String` 型にキャストされます
        return obj.length
    }

    // 型チェックが行われる分岐の外側では、`obj` は依然として `Any` 型である
    return null
}
//sampleEnd

fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?:
"Error: The object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf())
}

```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-is-operator"}

or:

```
//sampleStart
fun getStringLength(obj: Any): Int? {
    if (obj !is String) return null

    // この分岐では、`obj` は自動的に `String` 型にキャストされます
    return obj.length
}
//sampleEnd

fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?:
"Error: The object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf())
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-is-operator-expression"}
```

あるいは、次のように：

```
//sampleStart
fun getStringLength(obj: Any): Int? {
    // `&&` の右辺では、`obj` は自動的に `String` に型変換されます
    if (obj is String && obj.length >= 0) {
        return obj.length
    }

    return null
}
//sampleEnd

fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?:
"Error: The object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength("")
    printLength(1000)
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-basic-syntax-is-operator-logic"}
```

See [Classes](#) and [Type casts](#).